# "RAAHI-SMART BUS AND ROUTE MANAGEMENT SYSTEM"

*A*

***Project Report***

*submitted*

*in partial fulfillment*

*for the award of the Degree of*

***Bachelor of Technology***

***in Department of Information Technology***


**Project Mentor:**
Saroj Agarwal
Assistant Professor

**Submitted By:**

Saksham Agarwal,22ESKIT143
Khushi Jangid,22ESKIT302
Mohit Kumar 22ESKIT305



**Department of Information Technology**
**Swami Keshvanand Institute of Technology, M & G, Jaipur**
**Rajasthan Technical University, Kota**
**Session 2025-2026**

# Swami Keshvanand Institute of Technology, Management & Gramothan, Jaipur

**Department of Information Technology**

# CERTIFICATE

This is to certify that Mr Saksham Agarwal, a student of B.Tech(Information Technology) VIII semester has submitted her Project Report entitled ”Raahi-Smart Bus Route Management System” under my guidance

**Mentor**

Dr. Saroj Agarwal

Assistant Professor

Signature.....

**Coordinator**

Mrs. Priyanka Yadav

Assistant Professor

Signature.....

# Swami Keshvanand Institute of Technology, Management & Gramothan, Jaipur

**Department of Information Technology**

# CERTIFICATE

This is to certify that Ms Khushi Jangid , a student of B.Tech(Information Technology) VIII semester has submitted his Project Report entitled ”Raahi-Smart Bus Route Management System” under my guidance

**Mentor**

Dr. Saroj Agarwal

Assistant Professor

Signature.....

**Coordinator**

Mrs. Priyanka Yadav

Assistant Professor

Signature.....

# Swami Keshvanand Institute of Technology,

# Management & Gramothan, Jaipur

**Department of Information Technology**

# CERTIFICATE

This is to certify that Mr Mohit Kumar B.Tech(Information Technology) VIII semester has submitted his Project Report entitled ”Raahi-Smart Bus Route Management System” under my guidance

**Mentor**

Dr. Saroj Agarwal

Assistant Professor

Signature.....

**Coordinator**

Mrs. Priyanka Yadav

Assistant Professor

Signature.....

# DECLARATION

We hereby declare that the project report entitled ”Raahi: A Multi-Tenant Smart Transport Tracking and Safety Management System” is a record of original work carried out by us under the guidance of our project mentor in the Department of Information Technology, Swami Keshvanand Institute of Technology, Management and Gramothan, Jaipur. This project has been developed as partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Information Technology. To the best of our knowledge, this work has not been submitted elsewhere for the award of any other degree or diploma..

**Team Members** **Signature**

Saksham Agarwal, 22ESKIT143

Khushi Jangid, 22ESKIT302

Mohit Kumar, 22ESKIT305

# Acknowledgement

A project of such a vast coverage cannot be realized without help from numerous sources and people in the organization.We take this opportunity to express our gratitude to all those who have been helping us in making this project successful.

We are highly indebted to our mentor Ms.Shalini Singhal. She has been a guide, motivator source of inspiration for us to carry out the necessary proceedings for the project to be completed successfully. We also thank our project coordinator Ms. Priyanka Yadav for her co-operation, encouragement, valuable suggestions and critical remarks that galvanized our efforts in the right direction.

We would also like to convey our sincere thanks to Dr. Vipin Jain , HOD, Department of Information Technology, for facilitating, motivating and supporting us during each phase of development of the project.Also, we pay our sincere gratitude to all the Faculty Members of Swami Keshvanand Institute of Technology, Management and Gramothan, Jaipur and all our Colleagues for their co-operation and support.

Last but not least we would like to thank all those who have directly or indirectly helped and cooperated in accomplishing this project.

**Team Members**:

Saksham Agarwal, 22ESKIT143

Khushi Jangid, 22ESKIT302

Mohit Kumar, 22ESKIT305

# Abstract

Raahi is a web-based smart transport management and passenger safety platform designed to improve the efficiency, transparency, and security of institutional and city transport operations. The system provides role-based access for organization administrators, passengers, drivers, super administrators, and city passengers. It enables transport administrators to manage buses, routes, stops, drivers, and passengers from a centralized dashboard. Drivers can share live GPS location through their device, and passengers can track their assigned bus in real time through an interactive interface.

The platform also includes safety-oriented features such as SOS alerts, passenger issue reporting, guardian notification support, and route-level monitoring. A multi-tenant architecture is used so that each organization can securely operate within its own transport context using organization-specific codes and plan limits. The platform supports subscription-based scaling with trial, basic, pro, and enterprise plans, each enforcing usage limits on buses, passengers, and routes.

In addition to institution-focused transport management, Raahi includes a city passenger module where users can create accounts, search journeys between stops, detect nearest stops using geospatial queries, and select matching buses. The system is implemented using React, Vite, Express.js, MongoDB, Mongoose, Axios, and Leaflet. Raahi demonstrates how modern web technologies can be used to build a practical, scalable, and safety-focused transport solution for educational institutions and urban mobility scenarios.

# Table of Content

# List of Figures

# List of Tables

# Chapter 1
# Introduction

## 1.1 Problem Statement and Objective

Transportation management in colleges, schools, and urban passenger systems is often handled manually or through fragmented tools. This creates multiple operational issues such as poor route visibility, difficulty in assigning buses to passengers, inefficient communication between transport administrators and users, lack of real-time tracking, and delayed response in emergencies. In many cases, parents or guardians do not know the live location of the bus carrying their ward, and transport administrators have limited tools to monitor trip execution, route occupancy, or incident reporting.

The absence of an integrated system becomes even more problematic when the organization manages multiple buses, routes, drivers, and passengers. Manual record keeping makes the system error-prone, difficult to scale, and inefficient during urgent situations such as missed stops, overcrowding, or medical emergencies.

The objective of Raahi is to solve these problems by providing:

- centralized transport management for organizations
- real-time bus tracking for passengers
- route and stop management for administrators
- driver-side live location sharing
- subscription-aware scaling for different transport sizes

The major objective is to build a reliable and scalable digital platform that improves safety, transparency, and operational control in transport systems.

## 1.2 Literature Survey /Market Survey/Investigation & Analysis

The idea behind Raahi is inspired by problems observed in institutional transport systems and by trends in smart mobility platforms. Existing market solutions generally focus on one or two areas such as GPS tracking, route assignment, student attendance, or bus fleet monitoring. However, many solutions either lack affordability for small institutions, do not support organization-level isolation, or fail to combine safety, tracking, and administrative workflows into one unified platform.

The investigation of the problem domain showed the following common requirements:

- institutions need an easy interface for adding passengers, buses, drivers, and routes
- passengers need a simple way to see bus progress and estimated arrival
- guardians need fast notification during emergencies
- drivers need a lightweight interface to share location without complex setup
- system owners need organization-level separation and plan-based scaling

Based on code analysis, Raahi addresses these needs through a modular web platform rather than a single-purpose tracking application. It combines transport administration, live GPS-based monitoring, emergency alerting, issue escalation, and route intelligence in one system.

## 1.3 Introduction to Project

Raahi is a full-stack web application designed for smart transport administration and passenger safety. The frontend is built using React and Vite, while the backend is built using Express.js and MongoDB. The system supports multiple types of users:

- organization admin
- student or regular passenger

- driver
- platform super admin

The organization admin manages the transport ecosystem of a particular institution by creating passengers, buses, routes, and drivers. The driver shares live location using browser/device geolocation, and passengers track the bus through a map-based interface. Passengers can also raise emergency SOS alerts and submit issue reports. Super administrators can create organizations and manage stop and city route catalogs. City passengers can create accounts, search journeys, identify nearby stops, and select active buses serving their route.

The system follows a multi-tenant approach where organization-specific context is determined through an organization code. This allows the same backend to serve multiple institutions while keeping their transport data logically separated.

## 1.4 Proposed Logic / Algorithm / Business Plan / Solution

Raahi uses the following core logic:

- **Multi-tenant organization isolation :** The backend reads an organization code or organization ID from request context and binds all resource operations to that organization.
- **Subscription plan enforcement :** Before creating a bus, passenger, or route, the backend counts current records for the organization and checks limits based on plan type such as trial, basic, pro, or enterprise.
- **Automatic passenger-route-bus mapping :** When a student passenger is added, the system finds the route matching the passenger stop and automatically assigns the related running bus.
- **Live GPS tracking :** The driver device periodically sends latitude and longitude to the backend. The passenger-side interface fetches the latest location and displays it on a map.

- **Emergency notification workflow :** On SOS trigger, the system stores the event and attempts to notify the guardian through email, SMS, and WhatsApp depending on configured services.
- **Nearest stop detection :** The system uses geospatial stop data and MongoDB near queries to identify the closest active stops around the passenger location.
- **Issue reporting :** A passenger can report issues like wrong stop, crowding, or assistance requirement along with current location data.

## 1.5 Scope of the Project

The scope of Raahi includes:

- institutional transport digitization
- passenger registration and assignment
- bus, driver, route, and stop management
- real-time bus tracking
- issue reporting and emergency escalation
- multi-organization platform administration

The current scope does not fully include:

- payment integration
- native mobile application
- biometric attendance
- biometric attendance

# Chapter 2
# Software Requirement Specification

## 2.1 Overall Description

Raahi is a web-based transport management and safety information system. It allows educational institutions and transport operators to manage daily mobility through a centralized dashboard and role-specific workflows. It supports admin, passenger, driver, and super-admin usage scenarios. The system is intended to work in environments where buses, routes, and stops must be managed centrally while still providing end users with live tracking and emergency support features.

### 2.1.1 Product Perspective

Raahi is a modular client-server application. The frontend provides dashboards and user interaction screens. The backend exposes REST APIs for data management, location updates, alerts, and reporting. MongoDB is used for persistence, and external services such as SMTP and Twilio are used for notifications.

#### 2.1.1.1 System Interfaces

The technical integrity of the platform is maintained through a series of integrated software and communication interfaces that bridge the gap between the user and the data layer. The application utilizes a Decoupled Frontend-Backend Model, where a React-based user interface communicates with a Node.js/Express server via the Axios library for efficient state management and data fetching. Data persistence is facilitated through Mongoose, providing a structured schema-based solution to interact with the MongoDB NoSQL database.

Real-time spatial awareness is achieved by hooking into the Web Geolocation API on driver devices, which streams live coordinates to a Leaflet-powered mapping interface using OpenStreetMap tiles for high-fidelity visualization. To ensure the

system remains proactive rather than just reactive, it integrates external communication gateways: an SMTP-based Email Interface for formal guardian notifications and a Twilio API integration for instantaneous, high-priority alerts delivered through SMS and WhatsApp.

#### 2.1.1.2 User Interfaces

The system architecture features a modular front-end designed to cater to a hierarchical user base, ensuring that each stakeholder interacts with a tailored environment. The journey begins at a centralized Role Gateway, where users are routed to their respective modules based on authenticated permissions.

- **Administrative Ecosystems :** The Organization Admin Suite acts as a command center, utilizing a tabbed architecture to consolidate fleet logistics (buses and drivers), personnel management (students), and safety oversight (SOS alerts and incident reporting). For macro-level governance, the Super Admin and City Admin Dashboards provide tools for global infrastructure management, including organization onboarding, subscription modeling, and the standardization of city-wide transit stops and route geometries.
- **Operational Commuter Portals :** On the move, the Driver Interface provides a minimalist, high-utility dashboard focused on trip lifecycle management—allowing for vehicle selection and the synchronization of live telemetry. Simultaneously, the Passenger Experience is centered around a dynamic tracking module. This interface integrates live geospatial data to provide real-time map visualizations, predictive ETA updates, and a chronological route timeline. To prioritize safety, the commuter portal is equipped with immediate SOS triggers and one-touch reporting mechanisms.
- **Public Transit Integration :** Public Transit Integration: To accommodate general commuters, a specialized City Passenger Workflow handles the end-to-end user experience from account registration to journey planning. This includes intelligent search algorithms that suggest the most proximal boarding points and optimal ride selections based on the user's destination.

#### 2.1.1.3 Hardware Interfaces

The operational efficiency of the multi-organization management system relies on a distributed hardware network, ranging from centralized processing units to mobile edge devices. The infrastructure is categorized into three primary layers:

- **Centralized Hosting Infrastructure:** The core application logic and data persistence layers are deployed on a high-availability server or cloud-based environment. This machine acts as the central nervous system, managing concurrent database connections, processing API requests, and orchestrating the flow of information between different user roles.
- **Administrative and Command Stations:** High-level oversight and system configuration—performed by Super Admins and Organization Admins—are intended for use on desktop or laptop computing systems. These devices provide the necessary screen real-estate and processing power required for managing complex datasets, analyzing reports, and configuring multi-city route geometries.
- **Mobile Telemetry and End-User Devices:** Mobile Telemetry and End-User Devices: The system utilizes a ”Bring Your Own Device” (BYOD) approach for field operations. GPS-enabled smartphones serve as the primary hardware for drivers, utilizing onboard sensors to broadcast live geospatial telemetry. On the receiving end, passengers and guardians interact with the system through browser-enabled mobile devices or smartphones, which render the real-time tracking interface.
- **Connectivity Layer:** A fundamental dependency for the entire hardware stack is a stable and continuous internet connection. This network layer is critical for maintaining the heartbeat between the driver's telemetry and the cloud server, ensuring that live location updates and emergency SOS alerts are dispatched without latency.

#### 2.1.1.4 Software Interfaces

The system utilizes a modern, decoupled software stack designed for high performance, scalability, and real-time data processing. The integration of these technologies ensures a seamless flow from the database to the end-user interface.

- **Frontend Development Environment:** The client-side application is built using React, leveraging its component-based architecture for a modular user interface. Development is powered by Vite for optimized build speeds and Hot Module Replacement (HMR). To facilitate communication with the backend, Axios is utilized as the primary promise-based HTTP client, while Leaflet provides the core mapping engine for rendering geospatial data and route visualizations.

- **Backend and Logic Layer:** The server-side ecosystem is founded on Node.js, utilizing the Express.js framework to manage RESTful API endpoints and middleware. This layer handles the complex multi-tenancy logic, ensuring that data requests are filtered by organization and user role.

- **Data Persistence Layer:** For flexible and scalable data storage, the system employs MongoDB, a NoSQL database. Interaction between the server and the database is moderated by Mongoose, which provides a schema-based solution to model application data, ensuring strict data validation and efficient querying.

- **Communication External Gateways:** To bridge the gap between the digital platform and physical safety, the software integrates specialized notification interfaces. Nodemailer manages the SMTP handshake for automated email alerts to guardians, while the Twilio API is integrated to handle high-priority SMS and WhatsApp messaging for emergency SOS events.

- **Native Browser Integration:** The application maximizes the capabilities of modern web browsers through the Web Geolocation API, which captures real-time driver coordinates. Additionally, Local Storage is leveraged for persistent client-side session management, allowing for a faster user experience by caching non-sensitive user preferences and authentication tokens.

#### 2.1.1.5 Communications Interfaces

The system's architecture relies on a multi-channel communication strategy to ensure data synchronization, real-time tracking, and reliable alerting across different user tiers.

- **Synchronous Web Communication:** The primary exchange of information between the client-side interface and the server-side logic is facilitated through secure HTTP/HTTPS protocols. The system utilizes a RESTful API architecture, where the frontend dispatches requests to defined endpoints, and the backend responds with structured JSON (JavaScript Object Notation) payloads. This ensures a lightweight and standardized format for managing organization data, user profiles, and route configurations.
- **Real-Time Telemetry Flow:** A specialized communication stream is dedicated to Geospatial Telemetry. This involves a continuous uplink from the driver's mobile device to the backend server. As the driver moves, the device captures coordinates and transmits them via API requests, allowing the system to update the bus's position in the centralized database for live consumption by passengers.
- **External Notification Gateways:** For critical communication with stakeholders, the system integrates with external service providers. Formal notifications and non-urgent reports are dispatched via an SMTP-based Email Interface, providing a paper trail for guardians and admins. For time-sensitive safety alerts, such as SOS triggers, the system utilizes the Twilio API to bridge the gap between the web application and cellular networks, delivering instantaneous SMS and WhatsApp messages.
- **Geospatial Data Sourcing:** To provide visual context for the tracking data, the application establishes a communication link with third-party map providers. The system sends asynchronous Map Tile Requests to services like OpenStreetMap or CARTO. These services return the visual map layers required to render the route geometries and vehicle markers within the Leaflet-based

tracking interface.

#### 2.1.1.6 Operations

The functional lifecycle of the multi-organization management platform follows a structured sequence of operations, moving from high-level administrative setup to active real-time monitoring.

- **Phase 1: Administrative Initialization and Infrastructure Mapping**

  The process begins at the Super Admin level with the provisioning of a new organization and the assignment of a specific service plan. Once the organization is active, the local administrator defines the physical infrastructure by digitizing transit stops and route geometries. This creates the spatial foundation upon which the rest of the system operates.

- **Phase 2: Asset Management and Resource Allocation**

  With the routes established, the administrator populates the system with physical and human assets by registering vehicles (buses) and drivers. The system then facilitates a logical binding process, where buses are assigned to specific routes and designated drivers, ensuring a clear operational hierarchy for every trip.

- **Phase 3: User Onboarding and Intelligent Mapping**

  The final step of the setup phase involves passenger enrollment. To minimize manual configuration, the system utilizes an automated mapping algorithm that correlates passenger locations or requirements with existing routes and buses, effectively assigning them to the most relevant transit path.

- **Phase 4: Real-Time Trip Execution and Telemetry**

  The operational phase commences when a driver initiates a journey via the mobile interface. By triggering the location-sharing module, the driver's device begins a continuous stream of geospatial data to the backend. This allows the passenger tracking module to render a real-time visualization of the bus on a map, providing commuters with accurate ETAs and route progress.

- **Phase 5: Safety Oversight and Analytical Feedback**

  During an active trip, the system remains in a high-alert state, allowing passengers to submit incident reports or activate the SOS protocol in emergencies. The workflow concludes at the Administrative Dashboard, where all incoming alerts, safety reports, and trip data are aggregated into real-time analytics and monitoring views, providing the organization with a holistic overview of fleet performance and passenger safety.

#### 2.1.1.7 Project Functions

The system is engineered to provide a comprehensive suite of features distributed across multiple administrative and user layers. The core functionalities are categorized as follows:

- **Global Administration Governance:** The platform provides a Super Admin module dedicated to the high-level management of the ecosystem. This includes the ability to register new organizations, control their operational status, and enforce subscription-based limit checking to ensure resource compliance. This layer also governs the master data for city-wide transit, including the creation and administration of global stops and city-specific route networks.

- **Organizational Asset Personnel Management:** Within an individual organization, the system facilitates complete CRUD (Create, Read, Update, Delete) operations for fundamental assets. Administrators can manage the lifecycle of buses and drivers, performing logical resource binding by assigning specific drivers to vehicles and buses to designated routes. Personnel management extends to detailed passenger and guardian data synchronization, ensuring that every commuter is linked to an emergency contact profile.

- **Transit Engineering Geospatial Logic:** A core pillar of the platform is its ability to handle complex Route Management. This includes the dynamic creation and deletion of transit paths. For the public commuter, the system provides an intelligent City Passenger Flow, featuring secure authentication and a Geospatial Discovery Engine that allows users to search for journeys and

receive automated suggestions for the most proximal boarding points based on their current coordinates.

- **Real-Time Telemetry Safety Protocols:** The operational heartbeat of the system is the GPS Telemetry Engine, which handles the persistent logging of driver coordinates and the high-frequency retrieval of live locations for tracking. To prioritize safety, the system includes an Emergency Response Module capable of generating SOS events. When triggered, this module orchestrates instantaneous Guardian Notifications via integrated communication channels. Furthermore, passengers can file location-tagged issue reports, providing granular feedback on transit quality or safety concerns.
- **Intelligence Monitoring:** The system culminates in a Centralized Analytics Dashboard. Here, administrators can monitor the health of the organization through real-time alert tracking, incident reporting summaries, and statistical analytics, providing a data-driven overview of the entire transit operation.

#### 2.1.1.8 User Characteristics

The platform is designed to serve a diverse demographic of users, ranging from high-level system administrators to non-technical commuters. To ensure a seamless experience across this spectrum, the system adopts a role-centric design philosophy that minimizes cognitive load and technical barriers.

- **Platform Super Administrator:** This user possesses a high level of technical authority and is responsible for the macro-governance of the system. Their expertise allows them to manage multi-tenant configurations, subscription tiers, and city-wide infrastructure. The interface for this role prioritizes data density and comprehensive control over global settings.
- **Organizational Transport Administrator:** Typically an office-based professional, this user possesses foundational computer literacy. Their primary focus is on fleet logistics, personnel management, and incident response. To assist them, the dashboard utilizes familiar tabbed navigation and visual analytics, reducing the need for specialized technical training.

- **The Transit Operator (Driver):** This user role is characterized by a mobile-first environment. Drivers are expected to have basic smartphone proficiency. The interface is intentionally minimalist, featuring large touch-targets and streamlined workflows (such as a one-tap "Start Trip" button) to ensure they can manage location broadcasting without distraction during transit.
- **Institutional Commuter (Student/Employee):** These users typically have minimal technical prerequisites. Their interaction is focused on the consumption of information—specifically live tracking and safety features. The interface emphasizes clarity through visual map icons, progress timelines, and prominent SOS buttons, ensuring that even the most non-technical users can navigate the tracking portal intuitively.
- **Public City Passenger:** Representing the general public, these users seek immediate utility, such as route guidance or stop suggestions. Their persona requires a "low-friction" onboarding process, characterized by simple account creation and an intuitive search-based interface that mirrors popular commercial navigation apps.

#### 2.1.1.9 Constraints

For the multi-organization management platform to function optimally, several technical and environmental constraints must be acknowledged. These factors define the operational boundaries of the current implementation:

- **Network and Connectivity Dependency:** The system's core value proposition—real-time tracking and instantaneous alerting—is inherently dependent on persistent internet connectivity. Both the driver's telemetry broadcast and the passenger's live monitoring interface require a stable data link. Any network latency or disconnection will result in a "dead zone" where location updates and notifications cannot be synchronized.
- **Hardware and Permission Protocols:** The accuracy of the transit data relies on the GPS-enabled hardware of the driver's device. A critical operational constraint is the requirement for explicit user permissions; the system cannot

broadcast location data unless the driver provides the necessary geospatial authorizations within their browser or device settings.

- **Integration with Third-Party Gateways:** The automated communication layer is contingent upon the availability and valid configuration of external service credentials. Specifically, the dispatch of email alerts and SMS/WhatsApp notifications is tied to the operational status of the SMTP server and Twilio API account. In the absence of valid credentials or API credits, the notification module will remain inactive.

- **Architectural Contextualization:** Because the system is designed for multi-tenancy, a mandatory constraint for all organization-scoped APIs is the provision of a valid organization context. Requests made to the backend must include the specific organization identifier to ensure data isolation and prevent cross-tenant information leaks.

  Authentication a

- **Identity Framework:** The current iteration of the platform utilizes a lightweight authentication model. Rather than a complex JWT (JSON Web Token) or OAuth2 infrastructure, identity management is governed by a combination of role selection, organization codes, and administrative keys. This approach prioritizes rapid access and organizational simplicity over granular individual session management.

- **Geospatial Visualization Logic:** In the current frontend implementation, there is a distinction between bus position and route mapping. While the bus marker is driven by actual, real-time GPS telemetry, the underlying route visualization is partially simulated based on the sequence of defined stops. This ensures that the interface remains performant while providing a visual guide of the transit path.

#### 2.1.1.10 Assumption and Dependencies

The successful deployment and operational reliability of the multi-organization management system are predicated on a set of core assumptions and external technical dependencies.

The architectural design of the platform is built upon several key operational premises:

- **Data Siloing and Tenancy:** It is assumed that each organization operates as a distinct entity with an exclusive set of assets, including buses, passengers, routes, and drivers. The system relies on this isolation to maintain data integrity across the multi-tenant environment.
- **Credential Distribution:** The security and access model assumes that valid users have been pre-provisioned with the necessary organization codes and access keys. The platform operates on the premise that these identifiers are distributed through secure internal channels within each organization.
- **Device Availability and Permission:** For the tracking module to be effective, it is assumed that the driver's hardware maintains consistent access to location services. Furthermore, the system assumes that the driver will not manually revoke GPS permissions during the lifecycle of an active trip.
- **Backend Configuration:** The system assumes a correctly provisioned production environment, where environment variables (such as API keys and database strings) and the MongoDB connection are optimized for high-frequency data throughput.
- **Administrative Oversight:** It is assumed that organization administrators will actively maintain the system's accuracy by updating route geometries and vehicle assignments as physical transit schedules change.

# Chapter 3
# System Design Specification

## 3.1 System Architecture

Raahi follows a client-server architecture with modular separation between presentation, business logic, and data layers.

- **Presentation Layer :** Built using React. It includes admin dashboards, passenger tracking UI, driver tracking UI, and super-admin panels.
- **Application Layer :** Built using Express.js. It handles routing, validation, organization context resolution, plan checks, passenger assignment, live tracking, SOS, and issue reporting.
- **Data Layer :** Built using MongoDB and Mongoose. It stores organizations, passengers, guardians, buses, routes, stops, location logs, drivers, SOS records, reports, and city passenger accounts.
- **External Service Layer :** Includes SMTP for email alerts, Twilio for SMS/WhatsApp, and map tile services for visualization.

### 3.1.1 Module Decomposition Description

The system architecture is partitioned into eleven distinct modules, each responsible for a specific domain of the platform's logic. This modular approach ensures high maintainability and clear separation of concerns across the multi-organization environment.

1. **Identity Gateway Modules** : Serves as the primary entry point for the application. It manages role-based routing and implements security checks by validating organization-specific codes and administrative keys before granting access to internal dashboards.

2. **Organizational Resource Management** : Orchestrates the lifecycle of tenant organizations. It handles registration, status toggling (activation/deactivation), and enforces resource constraints (plan limits) on the total number of buses, routes, and passengers allowed per organization. These modules handle the physical assets of an organization. They facilitate the creation and categorization of the fleet (Institutional vs. City) and maintain a strict mapping between drivers and their assigned vehicles to ensure operational accountability.

3. **Transit Engineering Geospatial Logic** :

   Provides the tools for defining the transit network. It allows admins to construct routes by plotting start points, end points, and a sequence of intermediate stops, while managing the relational dependencies between routes and assigned assets. Maintains a geospatial catalog of stops using latitude and longitude coordinates. It supports advanced queries such as Nearest-Stop Suggestion and manages the public-facing ”City Passenger” experience, including account registration and journey matching.

4. **Operational Safety Modules** :

   The real-time heartbeat of the system. It synchronizes the Driver Geolocation Uplink with the Passenger Visualization UI, utilizing a persistent logging system in the backend to ensure the latest vehicle coordinates are always available for map rendering. A dedicated safety unit that monitors for emergency triggers and incident reports. Upon an SOS event, this module orchestrates multi-channel notifications to guardians via Email, SMS, and WhatsApp while archiving the alert history for administrative review.

5. **Intelligence Analytics** : An observational layer that aggregates raw data into actionable insights. It provides real-time counts of active assets and analyzes passenger distribution across different routes, helping organizations optimize their fleet utilization.

## 3.2 High Level Design Diagrams

### 3.2.1 Use Case Diagram

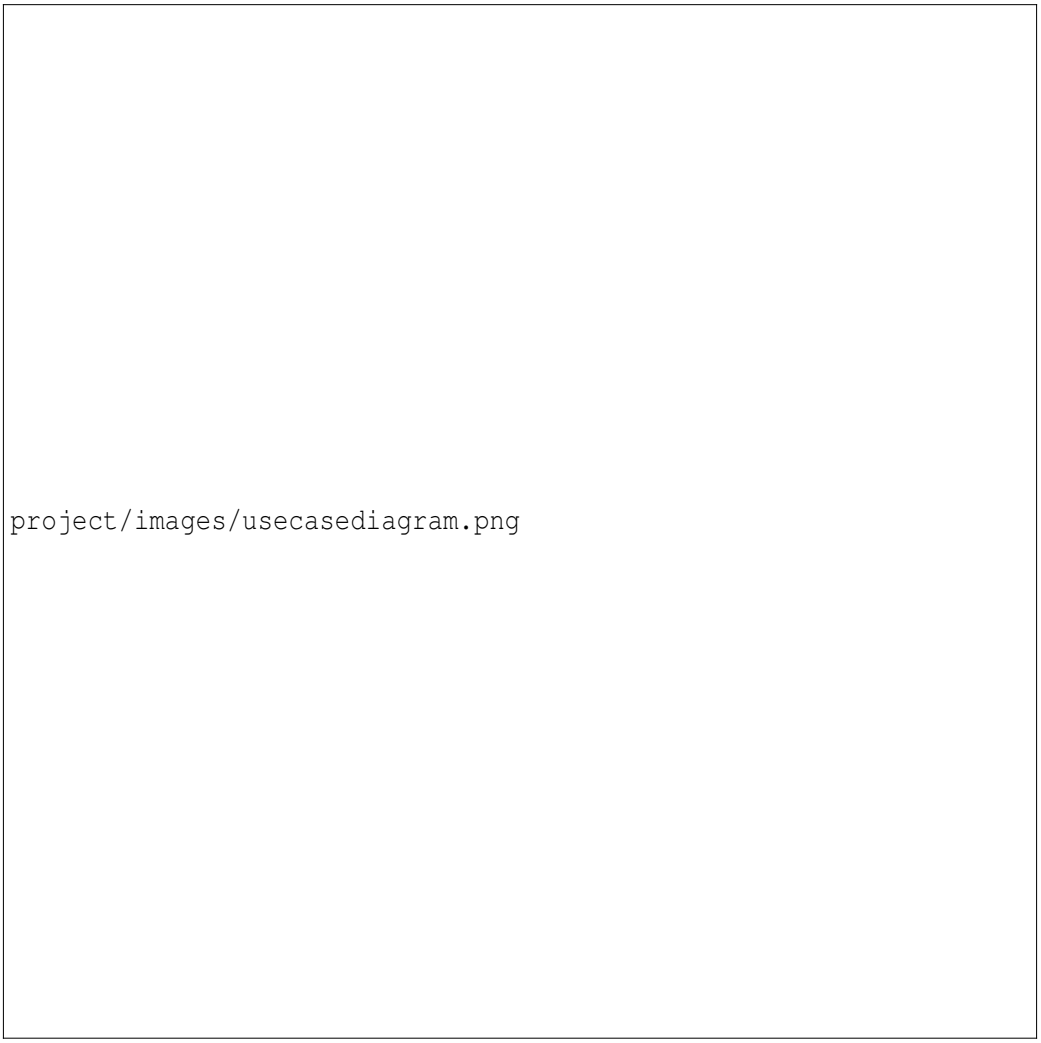


**Figure 3.1:** Use Case diagram

### 3.2.2 Activity Diagram

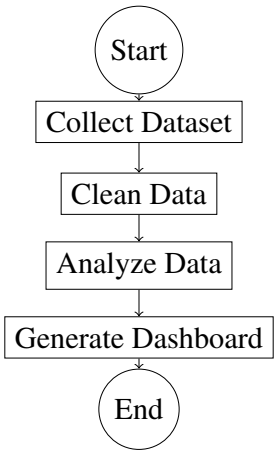


**Figure 3.2:** Activity Diagram

### 3.2.3 DFD Level 0

project/images/dfd1.png

**Figure 3.3:** DFD Diagram

### 3.2.4 ER Diagram

**Figure 3.4:** E-R Diagram

### 3.2.5 Class Diagram

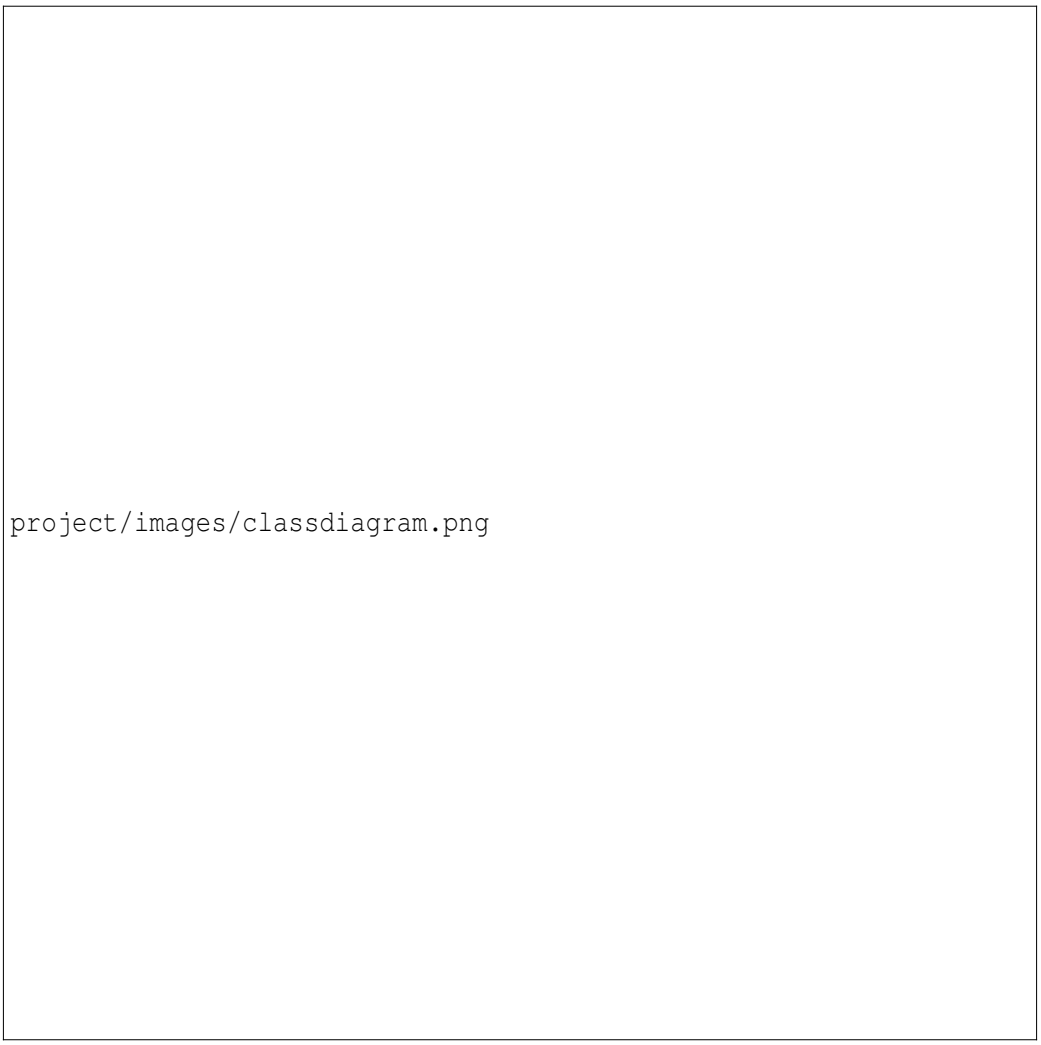


**Figure 3.5:** Class Diagram

### 3.2.6 Communication Diagram

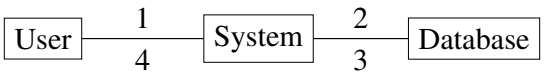


**Figure 3.6:** Communication Diagram

### 3.2.7 Sequence Diagram

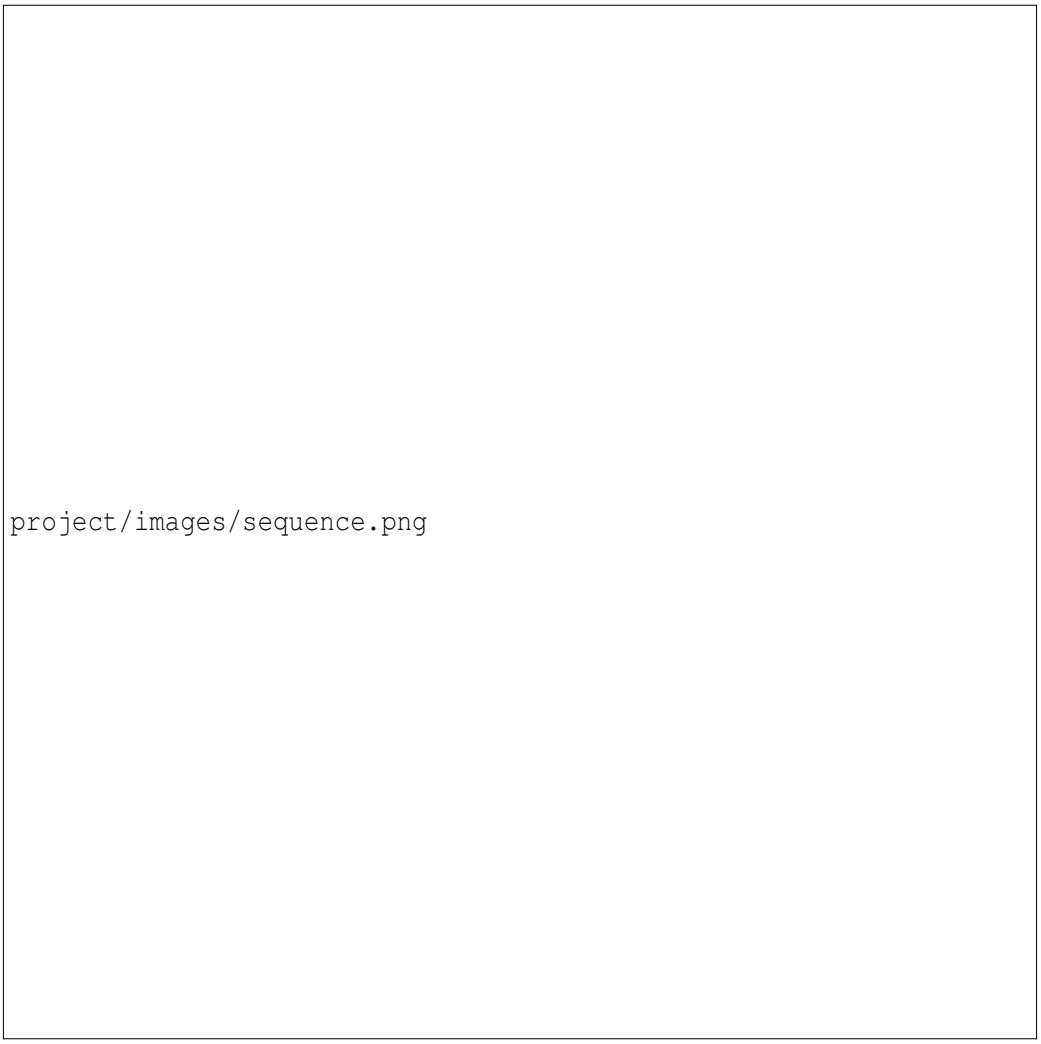


**Figure 3.7:** Sequence Diagram

### 3.2.8 Component Diagram

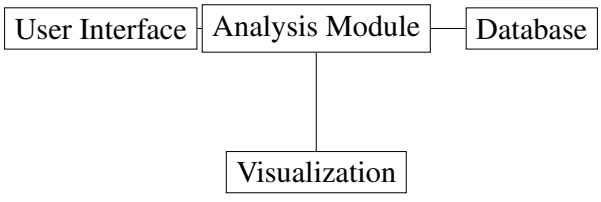


**Figure 3.8:** Component Diagram

### 3.2.9 Deployment Diagram

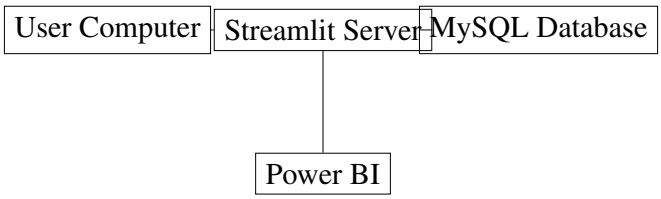


**Figure 3.9:** Deployment Diagram

### 3.2.10 Business Process Model

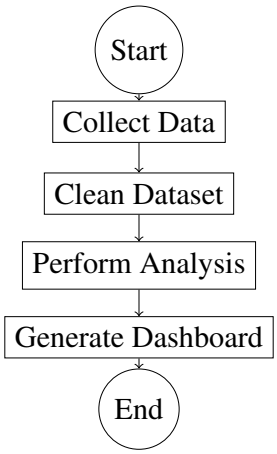


**Figure 3.10:** Business Process Model

# Chapter 4
# Methodology and Team

## 4.1 Introduction to Process Model (Waterfall Framework)

The Waterfall Process Model is one of the earliest and most traditional approaches used in software development. It follows a structured and sequential methodology in which the development process progresses through a series of well-defined stages. Each stage must be completed before moving to the next stage, and the output of one phase becomes the input for the following phase.

In this model, the development activities are arranged in a step-by-step flow similar to the movement of a waterfall. Because of this sequential structure, the model is simple to understand and easy to manage. Each phase focuses on a specific set of tasks, ensuring that the system is developed in an organized and controlled manner.

For the development of Raahi, the Waterfall Model was selected as the primary structural framework. In a multi-tenant environment where data isolation and security are paramount, a ”design-first” approach is essential. Unlike pure Agile, which can lead to ”scope creep” in database architecture, the Waterfall model ensured that the core relational logic between Organizations, Routes, and Buses was frozen and validated before the high-intensity frontend development commenced.

The ”Iterative Refinement” aspect was integrated during the Implementation phase, allowing for micro-sprints to fine-tune the real-time GPS polling intervals without disrupting the established backend schema.

## 4.2 Phases of the Waterfall Model

1. **Requirement Gathering and Analysis** : The lifecycle of Raahi commenced with a rigorous exploration of the challenges inherent in modern institutional

transit. This phase was characterized by stakeholder empathy and technical scoping, where we identified the critical disconnect between drivers, administrators, and commuters. We performed a detailed gap analysis, concluding that the primary requirement was not just a tracking app, but a multi-tenant governance framework. The outcome of this phase was a comprehensive Software Requirements Specification (SRS) that defined the functional boundaries of the system—specifically the need for data isolation via organization codes, real-time telemetry protocols, and automated emergency response systems. This phase ensured that every subsequent line of code addressed a verified real-world problem.

2. **System Design** : Once the requirements were finalized, the project transitioned into a high-level design phase where the conceptual was translated into the technical. We adopted a Decoupled Client-Server Architecture to ensure that the frontend and backend could scale independently. A significant portion of this phase was dedicated to ”Schema Engineering” within MongoDB. We designed a non-relational but highly structured data model that prioritized fast lookups for live coordinates while maintaining complex links between passengers, guardians, and routes. Simultaneously, the user interface was prototyped using a ”Role-Specific Design” philosophy, ensuring that the driver's interface was minimalist for safety, while the admin dashboard was data-dense for oversight.

3. **Implementation** : The implementation phase involved the actual construction of the platform using the MERN stack. This stage was divided into parallel development streams: the backend team focused on engineering RESTful API endpoints and the ”Auto-mapping” algorithm that links users to routes based on stop-name strings, while the frontend team utilized React and Vite to build a high-performance, single-page application. We integrated the Web Geolocation API to transform driver devices into active IoT sensors and used Leaflet.js to handle the heavy lifting of geospatial rendering. It was during this phase that we applied ”Iterative Refinement,” constantly tuning the frequency of API calls

to balance real-time accuracy with server performance and battery consumption on mobile devices.

4. **Integration and Testing** : With the code written, the project entered a critical validation phase. Integration testing was performed to bridge the gap between the isolated React components and the Express backend. We conducted ”End-to-End” (E2E) testing scenarios where we simulated a full transit cycle: from an admin creating a route to a driver broadcasting a location, and finally a passenger receiving that data. A major focus was placed on Safety Protocol Validation, where we intentionally triggered SOS alerts to verify the reliability of the Twilio and SMTP handshakes. We also tested the system's ”Multi-tenancy Integrity” by ensuring that data from ”Organization A” was never visible to ”Organization B,” confirming that our scoped API logic was impenetrable.

5. **Deployment** : The final phase of the Waterfall model involved preparing Raahi for a production-ready environment. This stage focused on ”Environmental Decoupling,” where we moved sensitive credentials like MongoDB connection strings and Twilio API keys into secure environment variables (.env files) to ensure the system could be deployed on any cloud infrastructure (such as AWS or Vercel) without code changes. Comprehensive technical documentation was produced, including an API directory and a user manual for organization admins. This phase concluded with a project retrospective, where we confirmed that the final deliverable was a stable, documented, and scalable solution that fulfilled every requirement identified in the first phase.

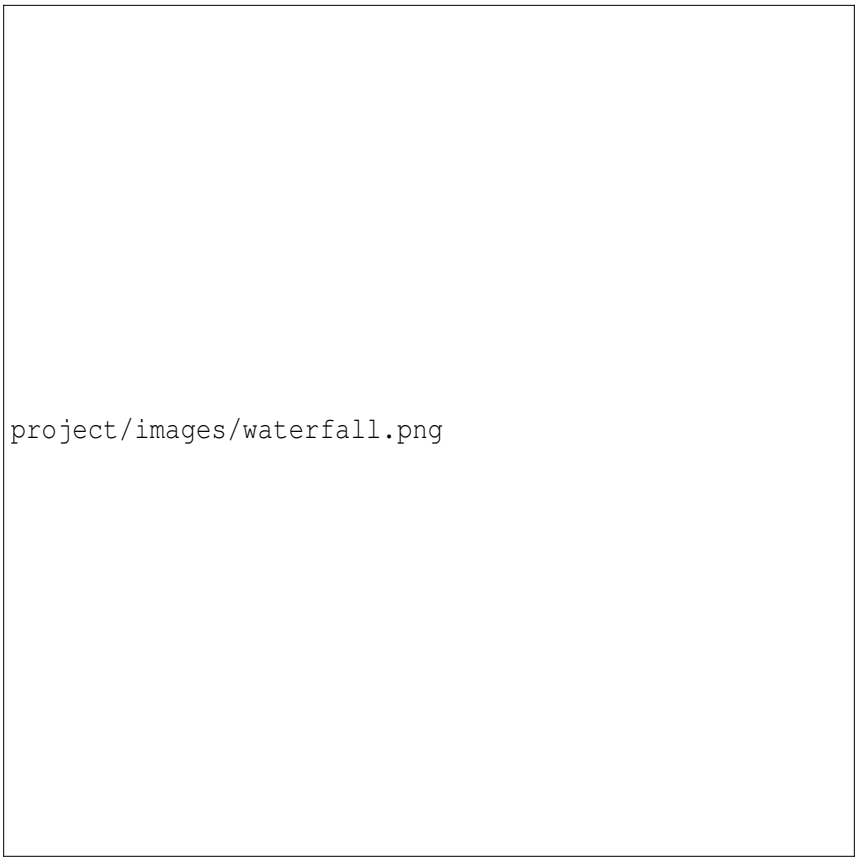


**Figure 4.1:** Waterfall Methodology

## 4.3 Advantages of the Waterfall Model

One of the main advantages of the Waterfall model is its simplicity and structured approach. Each phase has clearly defined objectives and deliverables, making the development process easier to manage. It also allows better planning because timelines and milestones can be defined for every stage of development. Since documentation is produced at each phase, it becomes easier for developers and stakeholders to understand the system.

## 4.4 Disadvantages of the Waterfall Model

Although the Waterfall model is straightforward, it has some limitations. One major drawback is that it does not easily accommodate changes once development has progressed to later stages. If new requirements arise after the design or implementation phase, modifying the system can be difficult and time-consuming. This model

is therefore more suitable for projects where requirements are stable and clearly defined from the beginning.

## 4.5 Team Members, Roles and Responsibilities

The successful completion of the Raahi - Smart Bus Route Management System project required collaboration among team members with different responsibilities. Each member contributed to specific tasks to ensure that the project was completed effectively.

1. **Saksham Agarwal:**
   The development of the Raahi platform was orchestrated through a strategic distribution of technical responsibilities, ensuring that the backend infrastructure, frontend visualization, and overall user flow were developed in parallel with high precision. Saksham Agarwal operated as the Lead Backend Architect, assuming full ownership of the server-side ecosystem and the project's data integrity. His work involved the foundational design of the MongoDB NoSQL schema, where he implemented a ”Scoped-API” logic to ensure that multi-tenant data remained strictly isolated between different organizations. Saksham was also the primary engineer behind the real-time telemetry engine, developing the logic for GPS coordinate logging and the complex integration of the Twilio and SMTP communication gateways. By managing the environment configurations and the administrative business logic, he provided the stable ”engine” required to power the high-frequency data demands of live bus tracking and emergency SOS dispatch.

2. **Khushi Jangid:**
   Simultaneously, Khushi Jangid served as the Lead Frontend and Integration Engineer, focusing on the ”Client-Side” execution and the visual rendering of the tracking telemetry. Her primary responsibility was the construction of the modular React environment, where she developed role-specific dashboards for admins, drivers, and passengers. A significant technical achievement of her role was the implementation of the Leaflet.js and OpenStreetMap interface, which

required optimizing state management to ensure that live bus markers could move across the map smoothly without requiring a full page refresh. Khushin acted as the bridge between raw data and user experience, utilizing Axios to synchronize the frontend with Saksham's backend APIs, ensuring that critical information like Estimated Time of Arrival (ETA) and route timelines were displayed with sub-second latency.

3. **Mohit Kumar:**

   The overall coherence and functional integrity of the platform were managed by Mohit, who filled the role of Workflow Designer and UI/UX Coordinator. Mohit's contribution was centered on the "User Journey Architecture," where he meticulously mapped out the transition states of the application—from the initial access selector page to the final city-passenger search flow. He was responsible for the visual consistency of the project, ensuring that the interface remained accessible to non-technical users such as bus drivers and elderly guardians. Beyond design, Mohit led the Quality Assurance and Documentation phase, conducting rigorous testing on the "Nearest Stop Suggestion" logic and the "City Journey Search" modules to identify and resolve logic bottlenecks. His efforts in compiling the technical documentation and coordinating the project flow ensured that the individual components developed by Saksham and Khushin were unified into a single, cohesive, and production-ready transit solution.

# Chapter 5
# Centering System Testing

## 5.1 Functionality Testing

Functionality testing for the Raahi platform was conducted with a rigorous focus on ensuring that every discrete module operates in strict alignment with the predefined user requirements. The primary objective was to validate the multi-tenant architecture, beginning with role-based access control to ensure that Super Admins, Organization Admins, Drivers, and Passengers are correctly routed to their respective environments. Significant testing was dedicated to organization resolution via unique codes and the integrity of CRUD operations for passengers, buses, and routes. We specifically validated the ”Intelligent Mapping Logic” that links commuters to transit paths, alongside the successful execution of the live location fetch cycle. Safety-critical features, such as the generation of SOS records and the subsequent trigger of guardian notifications via Twilio and SMTP, were prioritized to ensure zero-failure performance. Furthermore, we verified the geospatial search capabilities of the city passenger flow, confirming that the system accurately identifies the nearest stops and enforces plan-based resource limits across all active organizations.

## 5.2 Performance Testing

Performance testing for the current iteration of Raahi centered on practical responsiveness and the maintenance of a seamless user experience under typical institutional loads. We monitored the dashboard's ability to fetch and render datasets for passengers, buses, and routes, ensuring that data retrieval remained within acceptable latency thresholds. A critical performance metric was the five-second polling interval for live location updates; we verified that this frequent data fetching does not cause the passenger interface to freeze or the browser's main thread to become

unresponsive. Additionally, we analyzed the efficiency of the SOS and reporting modules, confirming that safety-sensitive actions are prioritized and complete with minimal server-side delay. While the current NoSQL structure and simple database count operations for plan-limit enforcement are highly efficient for small-to-medium deployments, we have identified that future enterprise-level scaling may necessitate the implementation of Redis-based caching, advanced indexing, and a transition to WebSocket-based tracking for even lower latency.

## 5.3 Usability Testing

The usability testing phase focused on the simplicity of interaction, recognizing that the system's primary users—including bus drivers and commuters—often require high-utility interfaces with low cognitive overhead. Our findings confirmed that the centralized role-selection gateway and the tab-based administrative navigation significantly reduce the learning curve for new users. The passenger tracking interface was specifically optimized to highlight the next stop, ETA, and SOS controls, ensuring that vital information is visible at a glance. For drivers, the workflow was intentionally restricted to three core actions—selecting a bus, starting a trip, and stopping a trip—to minimize distractions while driving. While the platform currently provides a high level of usability through direct-action design, our assessment highlights future opportunities for enhancement, including the implementation of even stronger role-based authentication protocols, more explicit visual loading indicators, and a comprehensive mobile-first optimization for the administrative dashboards to improve on-the-go management.

# Chapter 6
# Test Execution Summary

the rigorous verification of the Raahi platform using tools like Postman, MongoDB Atlas, and GPS-enabled devices. The testing suite confirmed the successful execution of 22 core test cases, validating multi-tenant data isolation, real-time telemetry accuracy, and the reliability of the SOS emergency protocols. Final observations indicate that the system is stable, with all functional modules performing according to specifications, making it ready for institutional deployment.

| S.No | Test Case ID | Test Case Description | Test Case Status | No. of Resources Consumed |
|---|---|---|---|---|
| 1 | TC01 | Create organization with unique code | Passed | 1 Tester |
| 2 | TC02 | Create organization with unique code | Passed | 1 Tester |
| 3 | TC03 | Resolve valid organization code | Passed | 1 Tester |
| 4 | TC04 | Login as organization admin using valid code | Passed | 1 Tester |
| 5 | TC05 | Add passenger with valid stop and guardian | Passed | 1 Tester |
| 6 | TC06 | Add passenger with invalid stop | Passed | 1 Tester |
| 7 | TC07 | Edit passenger details | Passed | 1 Tester |
| 8 | TC08 | Toggle passenger status | Passed | 1 Tester |
| 9 | TC09 | Add route with stops | Passed | 1 Tester |
| 10 | TC10 | Assign bus to route | Passed | 1 Tester |
| 11 | TC11 | Add driver and assign bus | Passed | 1 Tester |
| 12 | TC12 | Driver login with valid driver code | Passed | 1 Tester |
| 13 | TC13 | Driver starts trip sharing | Passed | 1 Tester |
| 14 | TC14 | Passenger tracks assigned bus | Passed | 1 Tester |
| 15 | TC15 | Passenger triggers SOS | Passed | 1 Tester |

| S.No | Test Case ID | Test Case Description | Test Case Status | No. of Resources Consumed |
|---|---|---|---|---|
| 16 | TC16 | Passenger submits quick issue report | Passed | 1 Tester |
| 17 | TC17 | Admin views SOS alerts | Passed | 1 Tester |
| 18 | TC18 | Admin views reported issues | Passed | 1 Tester |
| 19 | TC19 | City passenger finds nearest stops | Passed | 1 Tester |
| 20 | TC20 | City passenger searches and selects journey | Passed | 1 Tester |
| 21 | TC21 | Plan limit reached for trial/basic/pro | Passed | 1 Tester |
| 22 | TC22 | Super admin toggles organization status | Passed | 1 Tester |

**Table 6.1:** Test Execution Summary Report for Raahi-Smart Bus Route Management System

# Chapter 7

# Project Screen Shots

## 7.1 Project Screenshots

### 7.1.1 Login Overview

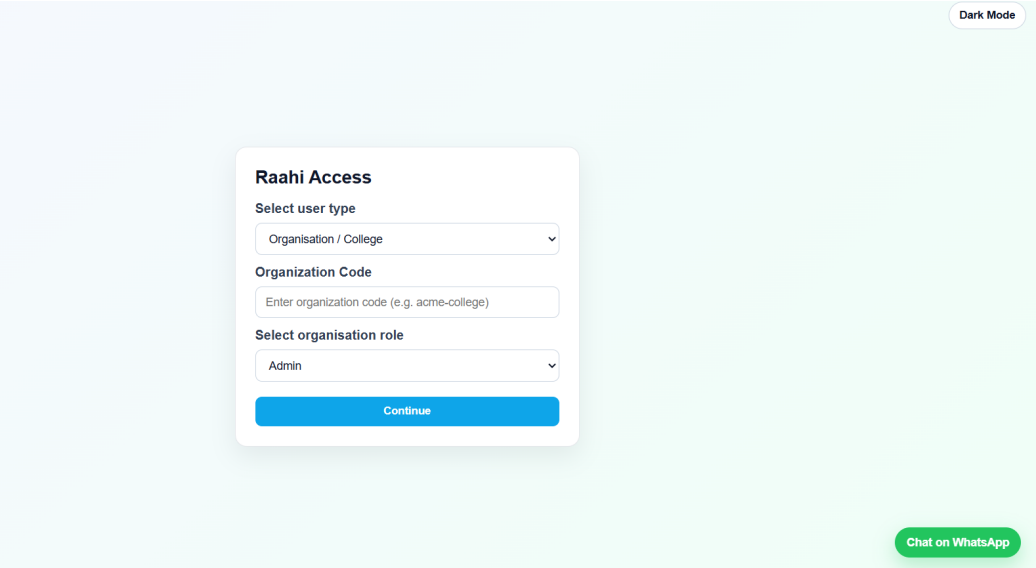


**Figure 7.1:** Raahi role selection interface

### 7.1.2 Organisation Admin Dashboard

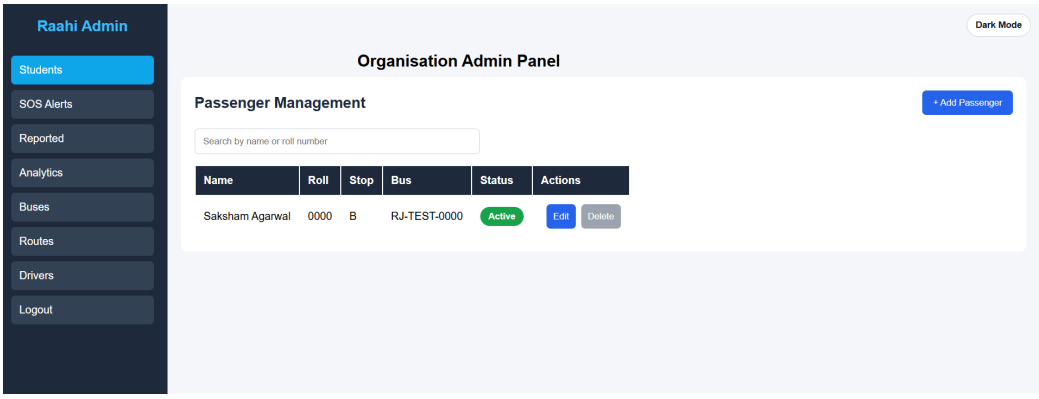


**Figure 7.2:** Organisation admin dashboard

### 7.1.3 Route Management Module

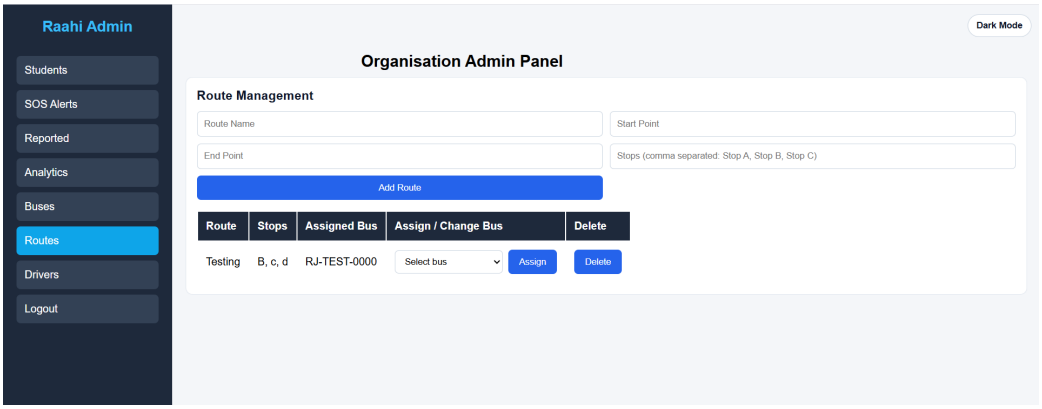


**Figure 7.3:** Route Creation and Bus Assignmet

### 7.1.4 Bus Management Module

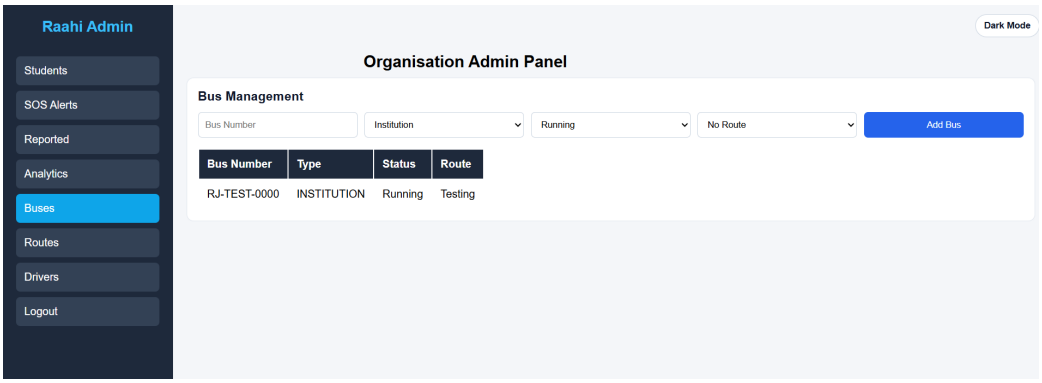


**Figure 7.4:** Bus management interface

### 7.1.5 Driver Management Module

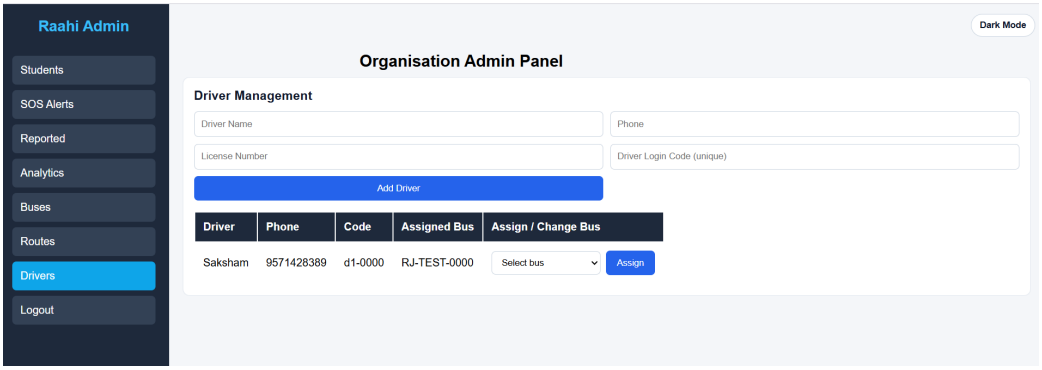


**Figure 7.5:** Driver creation and bus assignment

### 7.1.6 Student Live Tracking Map

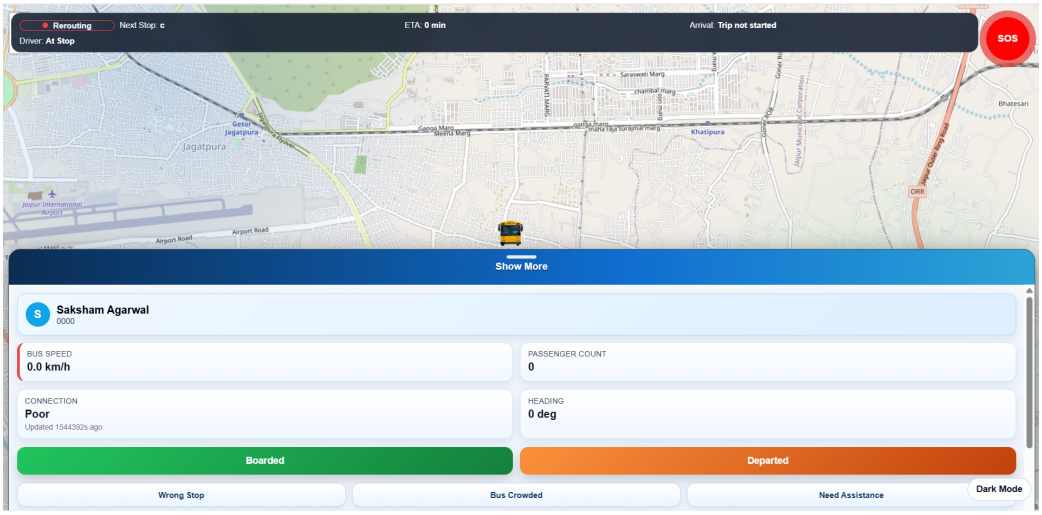


**Figure 7.6:** Passenger real-time bus tracking interface

### 7.1.7 SOS Alert Panel

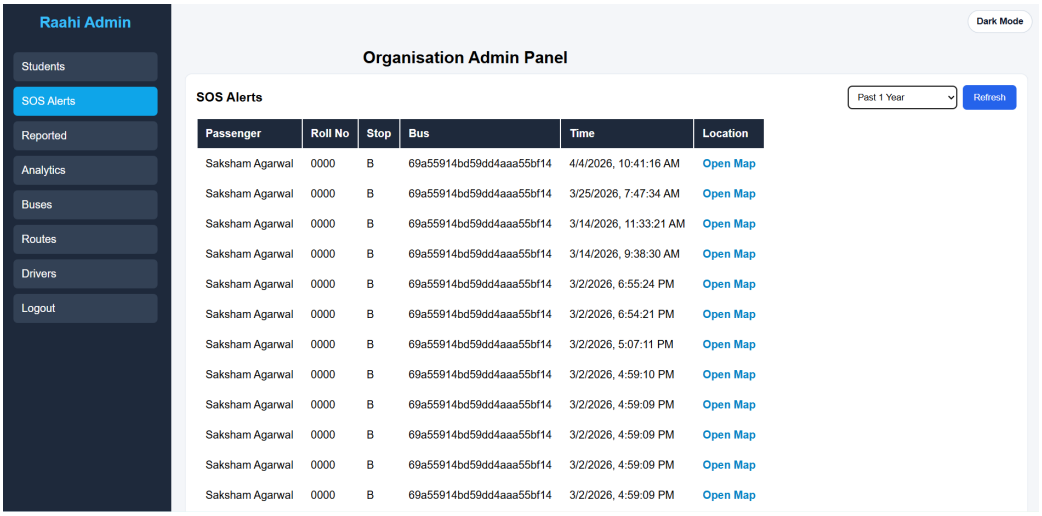


**Figure 7.7:** Admin view of SOS alert

### 7.1.8 Reported Issue Panel


**Figure 7.8:** Admin view of passenger issue report

### 7.1.9 Super Admin Organisation Panel

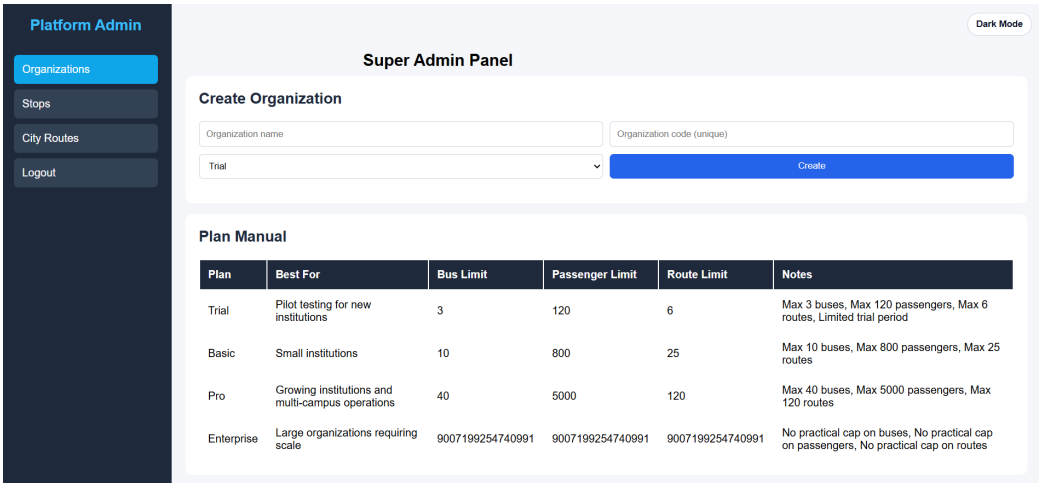


**Figure 7.9:** Super admin management of organisation and plans

# Chapter 8
# Conclusion and Future Scope

## 8.1 Conclusion

Raahi successfully demonstrates a practical smart transport management and passenger safety system built using modern web technologies. The project integrates multiple transport workflows into a single platform, including organization onboarding, passenger and route management, driver-based live tracking, emergency handling, issue reporting, and city transport assistance. A major strength of the project is its multi-tenant structure, which allows different organizations to use the same platform while preserving separate operational data and enforcing subscription limits.
The project also emphasizes safety and usability. Passengers can monitor bus movement, view route context, submit issue reports, and trigger emergency alerts with location details. Drivers can share location using a simple trip-control interface, and administrators can monitor alerts and manage transport resources from centralized dashboards. Overall, Raahi provides a strong foundation for a scalable transport-tech platform suitable for institutions and expandable to broader mobility environments.

## 8.2 Future Scope

The current implementation of Raahi establishes a versatile foundation for smart transit, but the architectural roadmap includes several high-impact enhancements designed to transition the platform from a prototype to an enterprise-grade mobility solution. A primary focus for future iterations is the hardening of system security through the transition to JSON Web Token (JWT) based authentication. This would replace the current lightweight access model with a stateless, encrypted identity management framework, allowing for granular role-based access control (RBAC) and significantly enhanced protection against unauthorized data access. Comple-

menting this security upgrade is the planned development of native Android and iOS applications, which will provide a more responsive user experience, background location processing for drivers, and the ability to leverage Firebase Push Notifications to alert passengers and guardians about delays or emergencies instantly.

In terms of geospatial intelligence and efficiency, the platform aims to move beyond simulated paths toward real-time route polyline generation. By utilizing advanced mapping APIs, the system will be able to render the actual road geometry between stop coordinates, providing a more accurate visual representation of the transit path. This data-rich environment will serve as the input for a sophisticated AI-based ETA Prediction Engine, which will analyze historical trip logs alongside real-time traffic data to provide commuters with highly accurate arrival estimates. Furthermore, the operational efficiency of the platform will be enhanced through QR-based passenger boarding, enabling automated attendance logging and providing administrators with real-time occupancy data. This will be paired with route optimization algorithms that suggest dynamic rerouting to drivers in response to roadblocks or heavy congestion, ensuring the fleet operates at peak efficiency.

To broaden the platform's commercial viability, future versions will incorporate integrated payment and ticketing support, particularly for city passengers. This module will facilitate secure mobile transactions and digital pass management, effectively transforming the platform into a full-service transit marketplace. Technical resilience will be further bolstered through offline caching mechanisms, utilizing service workers to ensure that tracking interfaces remain functional even in environments with intermittent network connectivity. Finally, the administrative experience will be elevated through advanced analytical exporting, allowing organizations to generate detailed PDF or CSV reports of trip performance, safety incidents, and asset utilization. These planned developments will collectively ensure that Raahi evolves into a comprehensive, AI-driven, and commercially scalable transit-tech ecosystem suitable for global deployment. In summary, the project highlights how integrating data analytics and visualization tools can help in understanding socio-economic development trends. By using technologies such as Python, MySQL, Power BI, and Next.js, the system provides valuable insights that can support.

# Chapter 9
# UN Sustainable Development Goals

## 9.1 Mapping with UN sustainable development goals

| Goal No. | Description | Project Contribution |
|---|---|---|
| 3 | **Good Health and Well-Being** | • The instantaneous SOS protocol ensures that safety or medical emergencies are communicated to guardians with precise GPS coordinates, drastically reducing the window for emergency intervention.<br>• The quick issue reporting module allows passengers to flag reckless driving or vehicle maintenance issues, facilitating administrative intervention before accidents occur.<br>• By providing transparent, real-time data on bus locations, the platform mitigates the mental stress and physical vulnerability associated with waiting in isolated areas for unverified transit. |
| 4 | **Quality Education** | • Institutional fleet synchronization ensures that students reach their educational institutions on time, preventing logistical delays from interfering with scheduled learning hours.<br>• The automated guardian notification system provides a transparent safety net, encouraging parents to enroll students in institutions that provide monitored and dependable transport.<br>• The route management module allows institutions to efficiently map and extend transport services to underserved or remote areas, ensuring that a student's location does not become a barrier to their education. |

**Table 9.1:** Mapping of Project Objectives with UN Sustainable Development Goals (SDG 3 and SDG 4)

# GitHub Link

## Link:

*https://github.com/saksham0208ag/raahi*

.....

# References

**Web and Technical References:**


- **React Documentation**- Used for developing the component-based frontend and managing real-time UI state updates. https://react.dev/
- **Express.js Documentation** —Utilized to architect the RESTful API and handle organization-scoped routing logic. https://expressjs.com/
- **MongoDB Documentation** — Referenced for implementing the non-relational database and managing multi-tenant data storage. https://www.mongodb.com/docs/
- **Mongoose Documentation** — Used to create structured schemas and provide validation for transit and user data. https://mongoosejs.com/
- **Leaflet Documentation** - Applied for integrating interactive maps and rendering live GPS markers for bus tracking. https://leafletjs.com/
- **Twilio Documentation** - Utilized to integrate high-priority SMS and WhatsApp messaging for emergency SOS alerts. https://www.twilio.com/docs/
- **Nodemailer Documentation** - Referenced for configuring secure SMTP email protocols for automated guardian notifications. https://nodemailer.com/


**Academic and Research References:**


- **Real-Time Bus Tracking System using IoT and Cloud Computing** https://ieeexplore.ieee.org/document/9072124
- **Smart Transport System with Emergency SOS and Real-Time Monitoring** https://www.researchgate.net/publication/342154678_Smart_Transport_System

- *Multi-Tenant Architecture for SaaS-Based Logistics Management*

  https://dl.acm.org/doi/10.1145/3384544

# Plagiarism Report

The project must be original and functional. The similarity index of the project report (including references) must be less than 20% overall and not more than 5% from any single source. The project report must contain 0% AI-generated content.

**You need to check plagiarism from chapter no-1 to chapter no-9 and references of report.**

# Published Paper/conference Certificate/Acceptance Letter

Here you need to provide certificate as proof that you participated in any of these activities. Hackathon or Research Paper publication/Presentation or Poster Competition or Patent filling.

- If you are a winner in any category such as in a Hackathon, poster competition, or research paper presentation , then provide the winner certificate.
- If your paper is yet not published but you have accepatance email then provide mail screenshot in this part.